University of Niš
Faculty of Sciences and Mathematics
Department of Computer Science
Doctoral Academic Studies in Computer Science

Seminar paper
Course: Artificial Neural Networks (20.IDI23)

Backpropagation for Feedforward Networks

Mentor
Prof. Branimir Todorović

Student
Elvir Muslić
ORCID 0009-0006-4042-6550

Niš, October 2026

Abstract

Backpropagation computes the gradient of a neural network's loss with respect to all of its weights in one pass backward through the network, at a cost of the same order as the forward pass. This paper presents it for feedforward networks of logistic and tanh units with half the mean squared error as the loss: the forward pass, the three rules that follow from the chain rule, the algorithm, and training by gradient descent. One step is worked by hand on a network with one hidden unit. PyTorch's automatic differentiation reproduces the hand gradient to within $1.4 \cdot 10^{-17}$, and central differences agree with it to within $2.9 \cdot 10^{-11}$. Trained with PyTorch by full-batch gradient descent, a network with four hidden units learns XOR, its loss falling from 0.1261 to $4.62 \cdot 10^{-4}$, and one with sixteen classifies all 200 training and all 200 test points of two interleaved half circles correctly.

Contents

1. Introduction 2
2. Feedforward Networks 3
3. Backpropagation 4
4. Training 6
5. Experiments 7
6. Conclusion 9

Code availability 9

References 10

1. Introduction

A feedforward network computes its output from its input through layers of simple units, each of which forms a weighted sum of its inputs and passes it through a fixed function. The weights are not designed but learned from examples: training changes them until a loss, which measures the error on the examples, is small. Gradient descent does this by moving every weight against the derivative of the loss with respect to it, so each of its steps needs the whole gradient, one derivative per weight.

Backpropagation computes all these derivatives in one pass backward through the network, from the output to the input, at a cost of the same order as the forward pass [1, 2]. It is the chain rule of calculus, organized so that each intermediate result is computed once. Rumelhart, Hinton and Williams [1] showed that it lets networks with hidden layers, whose units have no targets of their own, learn useful internal representations. Libraries such as PyTorch [3] now carry it out by automatic differentiation. The paper does four things.

(1) Section 2 describes the network, its forward pass and its loss.
(2) Section 3 derives the three rules of backpropagation from the chain rule, gives the algorithm, and works one step by hand.
(3) Section 4 explains training by gradient descent and checking gradients by central differences.

(4) Section 5 trains two small networks with PyTorch, after checking it against the step by hand.

2. Feedforward Networks

2.1. **The neuron.** A unit, or neuron, receives inputs $x_1, \dots, x_n$ and forms the weighted sum $a = w_1x_1 + \cdots + w_nx_n + b$. Each weight w_i says how strongly, and with which sign, its input counts, and the bias b shifts the sum, so that the unit can respond even when all its inputs are zero. The output of the unit is $h = \varphi(a)$, where the activation function φ is fixed.

This paper uses two smooth, S-shaped activations: the logistic function $\sigma(a) = 1/(1 + e^{-a})$, with values between 0 and 1, and $\tanh a$, with values between -1 and 1. Their slopes follow from their values, $\sigma'(a) = \sigma(a)\big(1 - \sigma(a)\big)$ and $\tanh'(a) = 1 - \tanh^2 a$, which the backward pass of Section 3 uses. This is why they must be smooth: a hard threshold, which jumps from one value to another, has slope zero wherever it has a slope, so the backward pass could carry nothing back through it.

The activation must be nonlinear. Without it, every layer would be an affine map, a weighted sum plus a constant; a chain of affine maps is again one affine map, so a network of any depth would compute no more than a single unit, which separates its inputs by a straight line (a flat boundary, for more than two inputs). Section 5.2 shows a problem where that is not enough.

2.2. **Layers and the forward pass.** A layer is a group of units that receive the same inputs. A feedforward network chains L layers, each feeding the next, with no loops: the input x enters layer 1, and the outputs of layer L are the output $\hat{y}$ of the network. Layer l has n_l units; its weights form an $n_l \times n_{l-1}$ matrix $W^{(l)}$, whose row j holds the weights of unit j, and its biases a vector $b^{(l)}$. With $h^{(0)} = x$, the forward pass computes, for $l = 1, \dots, L$,

$$a^{(l)} = W^{(l)}h^{(l-1)} + b^{(l)}, \qquad h^{(l)} = \varphi\big(a^{(l)}\big), \tag{2.1}$$

and returns $\hat{y} = h^{(L)}$; here φ acts on each component, and it may differ between layers. In words, each layer forms the weighted sums of the outputs of the previous layer and passes each sum through the activation.

Algorithm 1 states the forward pass step by step. It keeps every $a^{(l)}$ and $h^{(l)}$, which the backward pass needs. The layers between the input and the output are *hidden*: the examples give targets for the output only, not for them. A feedforward network of such layers with at least one hidden layer is called a multilayer perceptron [2]. A network with 2 inputs, one hidden layer of 4 units and 1 output unit is called a 2-4-1 network.

Algorithm 1 The forward pass

Input: input x; weights $W^{(l)}$ and biases $b^{(l)}$, $l = 1, \dots, L$

$h^{(0)} \leftarrow x$
for $l = 1, \dots, L$ **do**
  $a^{(l)} \leftarrow W^{(l)}h^{(l-1)} + b^{(l)}$ ▷ weighted sums (2.1)
  $h^{(l)} \leftarrow \varphi\big(a^{(l)}\big)$ ▷ outputs, kept for Algorithm 2
return $\hat{y} = h^{(L)}$

2.3. **The loss.** Training uses N examples, pairs (x_k, y_k) of an input and its target, $k = 1, \dots, N$ (here the subscript numbers the examples, not the components of an input). The loss scores the outputs $\hat{y}_k$ of the network against the targets:

$$E = \frac{1}{N} \sum_{k=1}^{N} \tfrac{1}{2} \|\hat{y}_k - y_k\|^2. \tag{2.2}$$

In words, E is half the mean of the squared errors over the examples; it is zero only if every output equals its target. Squaring makes errors of either sign count, and large errors count most. The factor $1/2$ cancels the 2 that differentiation brings down.

The loss depends on all the weights and biases through the forward pass. Since it is a mean, its gradient is the mean of the gradients of its terms, so Section 3 works with one example, $N = 1$, where $E = \frac{1}{2}\|\hat{y} - y\|^2$.

3. Backpropagation

3.1. **The chain rule and the delta.** A weight influences the loss only through a chain of intermediate values: it changes the weighted sum of its unit, which changes the output of the unit, which changes the weighted sums of the next layer, and so on up to $\hat{y}$ and E. The chain rule gives the derivative along such a chain as the product of the derivatives of its links; where a value influences E along several paths, the products of the paths add up. Computed weight by weight, these products would repeat the same links many times. Backpropagation computes each link once, starting from the loss and moving backward, and it keeps the results in one quantity per unit, the derivative of the loss with respect to the weighted sum of the unit:

$$\delta^{(l)} = \frac{\partial E}{\partial a^{(l)}}, \tag{3.1}$$

a vector with one component $\delta_j^{(l)}$ for each unit j of layer l. In words, $\delta_j^{(l)}$ says how much, and in which direction, the loss changes when the weighted sum of unit j changes. It serves as the error signal of the unit and is called its *delta*; Haykin [2] uses its negative, the local gradient.

Every weight and the bias of a unit act on E only through the weighted sum of the unit, so their gradients follow directly from its delta. The deltas of a layer, in turn, follow from those of the next layer.

3.2. **The three rules.** The two facts give three rules: one starts at the output, one sends the deltas one layer back, and one turns each delta into gradients.

Proposition 3.1 (The rules of backpropagation [1])**.** *For one example, with the forward pass* (2.1) *and the loss* $E = \frac{1}{2}\|\hat{y} - y\|^2$*, the deltas* (3.1) *and the gradients are*

$$\delta^{(L)} = (\hat{y} - y) \odot \varphi'\big(a^{(L)}\big), \tag{3.2}$$

$$\delta^{(l)} = \big(W^{(l+1)\mathsf{T}} \delta^{(l+1)}\big) \odot \varphi'\big(a^{(l)}\big), \qquad l = L-1, \dots, 1, \tag{3.3}$$

$$\frac{\partial E}{\partial W^{(l)}} = \delta^{(l)}\, h^{(l-1)\mathsf{T}}, \qquad \frac{\partial E}{\partial b^{(l)}} = \delta^{(l)}, \qquad l = 1, \dots, L, \tag{3.4}$$

where $\odot$ *multiplies two vectors component by component.*

Proof. All three follow from the chain rule. Since $a_j^{(L)}$ acts on E only through $\hat{y}_j = \varphi\big(a_j^{(L)}\big)$, and $\partial E/\partial \hat{y}_j = \hat{y}_j - y_j$, we get (3.2). For $l < L$, $a_i^{(l)}$ acts on E only through $h_i^{(l)}$, which enters each $a_j^{(l+1)}$ with the weight $W_{ji}^{(l+1)}$; summing over these paths gives $\delta_i^{(l)} = \varphi'\big(a_i^{(l)}\big) \sum_j W_{ji}^{(l+1)}\, \delta_j^{(l+1)}$, which is (3.3). Last, $W_{ji}^{(l)}$ enters only $a_j^{(l)}$, with $\partial a_j^{(l)}/\partial W_{ji}^{(l)} = h_i^{(l-1)}$, and $b_j^{(l)}$ enters it with the derivative 1, which gives (3.4). □

In words, the output delta is the error times the slope of the output unit; the deltas of each earlier layer are those of the next layer sent back through the same weights, transposed, times the slopes of its units; and the gradient of a weight is the delta of the unit it feeds times the input it carries, that of a bias the delta itself.

3.3. **The algorithm.** Algorithm 2 puts the rules in order for one example. It runs the forward pass, keeping every $a^{(l)}$ and $h^{(l)}$, then starts at the output with (3.2) and moves backward, one layer at a time, with (3.4) and (3.3). The slopes come from the kept outputs, $h^{(l)} \odot (1 - h^{(l)})$ for logistic units and $1 - h^{(l)} \odot h^{(l)}$ for tanh units, so nothing is computed twice. For N examples, the gradients of the single examples are averaged, as (2.2) is a mean.

Going backward is what makes the method cheap: each delta is computed once and serves both the weights that feed its unit and the deltas of the layer below. The forward pass needs one multiplication per weight, and the backward pass about two, one in (3.3) and one in (3.4), which multiplies each delta by each input, so it costs about twice as much as the forward pass, whatever the number of weights. Central differences, the alternative of Section 4, need two forward passes per weight.

3.4. **One step by hand.** Take the smallest network with a hidden unit: one input $x = 1$, one hidden and one output unit, both logistic, the weights $W^{(1)} = W^{(2)} = 0.5$, no biases, and the target $y = 1$. Table 1 follows the forward pass, the loss and Algorithm 2 line by line, in double precision. The gradient-descent step that uses these gradients, with $\eta = 1$, follows in Section 4.

The forward pass gives the output $\hat{y} = 0.577185380$, below the target, and the loss $E = 0.089386101$. The backward pass starts at the output: by (3.2), $\delta^{(2)}$ is the error $\hat{y} - y$ times the slope $\hat{y}(1 - \hat{y})$ of the output unit. Rule (3.3) sends it back through $W^{(2)}$ and multiplies it by the slope of the hidden unit, and (3.4) turns each delta into the gradient of the weight that feeds its unit.

Algorithm 2 Backpropagation for one example

Input: example (x, y); weights $W^{(l)}$ and biases $b^{(l)}$, $l = 1, \dots, L$

1: run Algorithm 1 on x, keeping $a^{(l)}$ and $h^{(l)}$, $l = 1, \dots, L$
2: $\delta^{(L)} \leftarrow (\hat{y} - y) \odot \varphi'\big(a^{(L)}\big)$ ▷ output delta (3.2)
3: **for** $l = L, L-1, \dots, 1$ **do**
4: $\quad \partial E/\partial W^{(l)} \leftarrow \delta^{(l)} h^{(l-1)\mathsf{T}}$, $\partial E/\partial b^{(l)} \leftarrow \delta^{(l)}$ ▷ gradients (3.4)
5: $\quad$ **if** $l > 1$ **then** $\delta^{(l-1)} \leftarrow \big(W^{(l)\mathsf{T}} \delta^{(l)}\big) \odot \varphi'\big(a^{(l-1)}\big)$ ▷ one layer back (3.3)
6: **return** the gradients $\partial E/\partial W^{(l)}$ and $\partial E/\partial b^{(l)}$, $l = 1, \dots, L$

TABLE 1. One step by hand, with $x = 1$, $y = 1$, $W^{(1)} = W^{(2)} = 0.5$, no biases and logistic units: the forward pass, the loss and the backward pass, each row by the rule in its first column, to nine decimals.

from	quantity	value
forward pass (2.1)	$a^{(1)} = W^{(1)}x$	0.500000000
	$h^{(1)} = \sigma\big(a^{(1)}\big)$	0.622459331
	$a^{(2)} = W^{(2)}h^{(1)}$	0.311229666
	$\hat{y} = \sigma\big(a^{(2)}\big)$	0.577185380
loss (2.2)	$E = \frac{1}{2}(\hat{y} - y)^2$	0.089386101
output delta (3.2)	$\delta^{(2)} = (\hat{y} - y)\,\hat{y}\,(1 - \hat{y})$	−0.103184702
gradient (3.4)	$\partial E/\partial W^{(2)} = \delta^{(2)}h^{(1)}$	−0.064228280
one layer back (3.3)	$\delta^{(1)} = \delta^{(2)}W^{(2)}h^{(1)}(1 - h^{(1)})$	−0.012124394
gradient (3.4)	$\partial E/\partial W^{(1)} = \delta^{(1)}x$	−0.012124394

Both gradients are negative, so increasing either weight lowers E and raises $\hat{y}$ toward the target.

On the way back, the delta shrinks: $\delta^{(1)}$ is $\delta^{(2)}$ times $W^{(2)} = 0.5$ and times a slope of at most $1/4$, the largest slope of the logistic function. Over many layers, this shrinking can slow down the learning of the first layers, a difficulty known as the vanishing gradient.

4. Training

Training looks for weights and biases with a small loss (2.2). Gradient descent starts from random values and repeats one step: Algorithm 2, run on all N examples, gives the gradient, and the weights and biases of every layer l move against it,

$$W^{(l)} \leftarrow W^{(l)} - \eta\,\frac{\partial E}{\partial W^{(l)}}, \qquad b^{(l)} \leftarrow b^{(l)} - \eta\,\frac{\partial E}{\partial b^{(l)}}, \tag{4.1}$$

with a step size $\eta > 0$, also called the learning rate. In words, every weight and bias takes a step downhill, of a length proportional to its own derivative.

So a small enough step lowers E, unless the gradient is zero. For a small step, the change of E is about the step times the slope of E along it, and along the negative gradient that slope is as steep as it gets: the step lowers E by about η times the sum of the squared derivatives. Too large a step can overshoot the valley and raise E; too small a step makes little progress.

On the network of Section 3.4, one step with $\eta = 1$ moves the weights to $W^{(1)} = 0.512124$ and $W^{(2)} = 0.564228$, the output rises to $\hat{y} = 0.587300$, and the loss falls from 0.089386 to 0.085161.

Gradient descent can at best reach a point where the gradient is nearly zero, which need not have the lowest loss. For a convex loss, one that never rises above the straight line between two of its points, a zero gradient would mean the lowest loss; but the loss of a network with hidden units is not convex in the weights. Exchanging two hidden units, with their weights, leaves the loss unchanged, so its minima come in copies, usually with a higher loss between them, which a convex loss cannot have. Gradient descent follows the

local slope only: it can stop in a local minimum or slow down on a nearly flat stretch, and where it ends depends on the initial weights. The experiments therefore fix the random seed.

A backpropagation code, by hand or in a library, can be checked against differences of the loss, which need no derivative at all. For one parameter θ_j, a weight or a bias, the central difference is

$$\frac{\partial E}{\partial \theta_j} \approx \frac{E(\theta_j + \epsilon) - E(\theta_j - \epsilon)}{2\epsilon}, \tag{4.2}$$

where $E(\theta_j \pm \epsilon)$ is the loss with θ_j changed by $\pm\epsilon$ and every other parameter fixed. In words, the slope is measured by moving one parameter a little both ways and dividing the change of the loss by the distance. The error of (4.2) shrinks like ϵ^2, but the rounding errors of the two losses are divided by 2ϵ, so ϵ cannot be too small. A whole gradient costs two forward passes per parameter, which makes (4.2) a check, not a way to train.

5. Experiments

PyTorch [3] is the method, and the step by hand of Section 3.4 is the check. The notebook builds and trains the networks with the layers, the automatic differentiation and the gradient descent of PyTorch, which match the paper as follows.

(1) `nn.Linear` holds the weights and biases of a layer and computes its weighted sums, and `nn.Tanh` and `nn.Sigmoid`, the logistic function σ, apply the activations; `nn.Sequential` chains the layers into Algorithm 1, the forward pass (2.1).
(2) `0.5 * ((model(X) - Y) ** 2).mean()` is the loss (2.2) for one output unit.
(3) `loss.backward()` computes the gradient by automatic differentiation (autograd): PyTorch records the operations of the forward pass and applies the chain rule to them in reverse order. For these networks, this is Algorithm 2 on all examples at once, with each delta divided by N, since (2.2) is a mean.
(4) `optimizer.step()` of `torch.optim.SGD` applies the update (4.1).

All computations are in double precision, which the comparison of Section 5.1 needs. Both experiments train a network with tanh hidden units and a logistic output unit, whose values between 0 and 1 suit the targets 0 and 1, by full-batch gradient descent, each step on all N examples, with $\eta = 1$. They start from the default initial weights and biases of PyTorch, which for a layer with n inputs are uniform on $[-1/\sqrt{n}, 1/\sqrt{n}]$, drawn after `torch.manual_seed(0)`. An output above 0.5 counts as class 1.

5.1. **The hand step against autograd and central differences.** Autograd reproduces the hand gradient of Table 1 to rounding, and central differences to the accuracy that (4.2) allows. For this check, the notebook writes the loss of the network of Section 3.4 as a function of its two weights and lets PyTorch differentiate it at $W^{(1)} = W^{(2)} = 0.5$. Its two derivatives differ from the hand gradient by at most $1.4 \cdot 10^{-17}$, a rounding error.

The central differences (4.2) with $\epsilon = 10^{-6}$ differ from the hand gradient by $2.2 \cdot 10^{-12}$ for $W^{(1)}$ and $2.9 \cdot 10^{-11}$ for $W^{(2)}$, far above the rounding error of

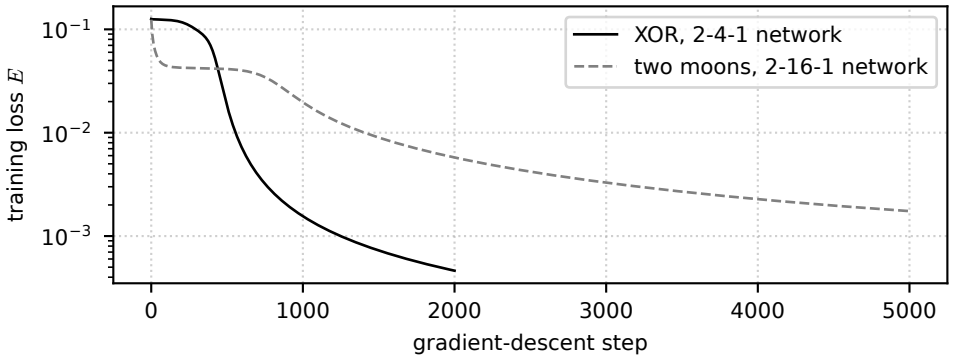


FIGURE 1. Training loss E of (2.2) at every step of full-batch gradient descent with $\eta = 1$: the 2-4-1 network on XOR (solid) and the 2-16-1 network on the two moons (dashed).

automatic differentiation and far below the size of the gradients: the rounding errors of the two losses, divided by 2ϵ, limit their accuracy.

5.2. **XOR.** Gradient descent teaches a 2-4-1 network the XOR function, which no network without hidden units can represent. XOR maps the inputs $(0, 0)$, $(0, 1)$, $(1, 0)$ and $(1, 1)$ to the targets 0, 1, 1 and 0. No straight line separates the two inputs with target 1 from the two with target 0 (Section 2.1). The network has 17 parameters and is trained on the four examples for 2000 steps.

The loss falls from 0.1261 to $4.62 \cdot 10^{-4}$ (Figure 1), and the outputs for the four inputs are 0.03, 0.98, 0.97 and 0.04, so all four are classified correctly. The loss falls at every step, but not at an even pace: after 200 steps it is still 0.1174, by step 500 it is 0.0181, and then it falls slowly again. This is a nearly flat stretch of the kind that Section 4 describes, which this run leaves after a few hundred steps.

5.3. **Two moons.** A 2-16-1 network separates two interleaved half circles, and its boundary classifies new points from the same source correctly as well. The data come from `make_moons` of scikit-learn: the two half circles, class 0 above and class 1 below, with Gaussian noise of standard deviation 0.1. No straight line separates the two arcs. The training and the test set, 200 points each, are drawn with the seeds 0 and 1; they share the points without noise and differ only in the noise. The network has 65 parameters and is trained for 5000 steps.

The loss falls from 0.1241 to 0.0017 (Figure 1), again at every step: fast to 0.0444 within 100 steps, then hardly, to 0.0414 at step 500, and then faster again, to 0.0197 by step 1000. The trained network classifies all 200 training points and all 200 test points correctly. Its decision boundary, the curve on which the output equals 0.5, runs through the gap between the two arcs (Figure 2).

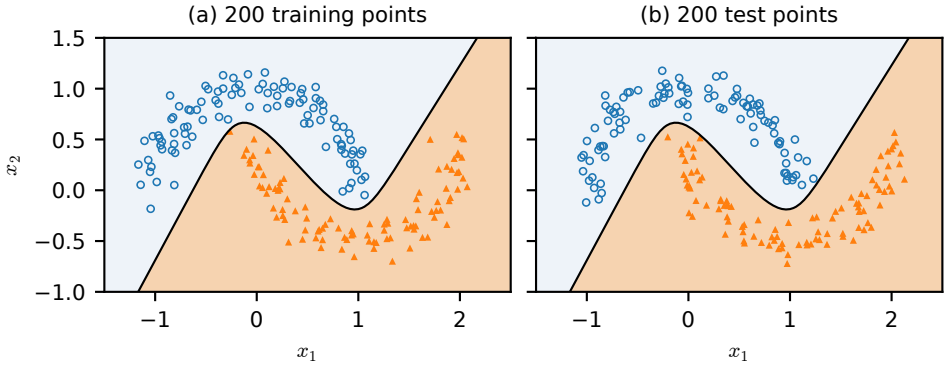


FIGURE 2. The two moons and the decision boundary of the trained 2-16-1 network, the curve on which its output equals 0.5, with the region of class 1 shaded darker: (a) the 200 training points, (b) the 200 test points. Open circles: class 0; filled triangles: class 1.

6. Conclusion

(1) Backpropagation is the chain rule, organized backward: three rules (Proposition 3.1) give all the gradients in one backward pass, which costs about twice the forward pass.
(2) On the network of Section 3.4, the automatic differentiation of PyTorch reproduces the hand gradient to within $1.4 \cdot 10^{-17}$, and central differences to within $2.9 \cdot 10^{-11}$.
(3) A 2-4-1 network learns XOR: its loss falls from 0.1261 to $4.62 \cdot 10^{-4}$, and all four outputs are on the right side of 0.5.
(4) A 2-16-1 network classifies all 200 training points and all 200 test points of the two moons correctly.

The findings have limits: two small problems, one run each from one seed, plain full-batch gradient descent with one step size, and a test set that differs from the training set only in its noise. They illustrate the method rather than measure how well it works in general. Faster training methods change the step, not the gradient, which backpropagation still computes. Applied to a recurrent network unrolled along its input sequence, the same recursion gives backpropagation through time [4]. Recurrent networks, whose training one of the course texts [5] treats as the estimation of parameters, states and structure, are outside this paper.

Code availability

The notebook `backprop_project.ipynb`, in the folder `project`, builds and trains the networks with PyTorch and computes every number, table and figure in Sections 3.4, 4 and 5. It runs on Python 3.14.7 with PyTorch 2.14.1, scikit-learn 1.9.1 and Matplotlib 3.11.2. The three library versions, and NumPy, are pinned in `project/requirements.txt`; with them installed in a virtual environment `project/.venv` (`cd project`, `python3 -m venv .venv`, then `.venv/bin/pip install -r requirements.txt`), the command

```
.venv/bin/jupyter nbconvert --to notebook --execute --inplace backprop_project.ipynb
```

runs it; the run reported here took a few seconds. The public repository https://github.com/innicoder/backprop-seminar holds this paper, its LaTeX source, the notebook with its outputs, and the figures.

References

[1] D. E. Rumelhart, G. E. Hinton, and R. J. Williams. "Learning representations by back-propagating errors". In: *Nature* 323.6088 (1986), pp. 533–536. DOI: 10.1038/323533a0.

[2] S. Haykin. *Neural Networks: A Comprehensive Foundation*. 2nd ed. Upper Saddle River, NJ: Prentice Hall, 1998.

[3] A. Paszke, S. Gross, F. Massa, A. Lerer, et al. "PyTorch: An imperative style, high-performance deep learning library". In: *Advances in Neural Information Processing Systems 32*. 2019. arXiv: 1912.01703.

[4] P. J. Werbos. "Backpropagation through time: what it does and how to do it". In: *Proceedings of the IEEE* 78.10 (1990), pp. 1550–1560. DOI: 10.1109/5.58337.

[5] B. Todorović, S. Todorović-Zarkula, and M. Stanković. *Rekurentne neuronske mreže: estimacija parametara, stanja i strukture*. Niš: Univerzitet u Nišu, Prirodno-matematički fakultet, 2012.